
Prism: Unlocking Language Model Capability Extraction

Abhishek Mishra

Krishna Pagare

Abstract

A trained language model has many capabilities at once, yet a single deployment usually exercises one of them. It still pays for the compute cost of all the rest. Quantization and pruning do not change this. They shrink the whole model, often require retraining, and operate on weights rather than on capabilities. Neither of the systems maintains a single capability while shedding the cost of the others.

We ask whether one capability can be separated without retraining a replacement model. We call a capability extractable when a chosen subset of MLP channels can carry it on its own. The remaining channels remain deactivated and the surrounding transformer remains unchanged. The retained channels form a substrate, and recovery measures how much of the original behavior the substrate preserves. The result is not a compressed model. It is evidence that the substrate is sufficient under the stated intervention.

How small that substrate can be depends on a prior step we call *collimation*, an adaptation that moves where a behavior is carried while leaving the model's outputs unchanged. Collimation before channel selection shrinks the sufficient substrate, whereas collimation after it only repacks the behavior inside a fixed budget. This ordering separates collimation from fine-tuning, since training harder would not depend on selection order.

On a 1.5B math model, the arithmetic recovery that once required about 90% of MLP channels drops to 5% after collimation, an 18-fold reduction. In open 8B weights, the function calling increases from 19.1% to 84.6% recovery on a fixed 36.2% substrate.

Keywords: capability extraction, model sparsity, model efficiency, knowledge distillation, low-rank adaptation, function calling, machine translation.

1 Introduction

Modern model efficiency rests on a simple hope. Much of a large model is not needed for any particular behavior. Pruning, sparse training, distillation, low-rank adaptation, and conditional computation each exploit some version of that hope. They usually ask a model-level question, namely how much capacity can be removed, skipped, or approximated while aggregate performance stays acceptable. We ask the capability-level version instead. Given a named behavior, can we make the model expose a small substrate that still runs that behavior once the surrounding capacity is switched off?

This raises a question we can check with a clear success criterion. It is not enough that a channel looks useful under attribution. What matters is whether the channels we keep, taken together, still carry the behavior once the rest of the model is switched off. That turns extraction into a problem of causal discovery rather than scoring, where we look for the part of the network that genuinely represents the ability, not the part that merely correlates with it. That part is not always laid out cleanly along the model's existing channels, which is the difficulty the rest of this paper confronts.

The MLP channels we keep are the *mask*, and we switch off the rest under a *contract* that sets how they go dark and how we score what survives. The mask does its work inside the *scaffold*, the attention, embeddings, layer norms, and unembedding we leave in place, and the kept channels carrying the behavior there are the *substrate*. When the behavior holds under that contract, the mask is *causally sufficient*. A mask that only correlates with the behavior shows where signal flows, not where the capability can run.

Two-digit addition in Qwen2.5-Math-1.5B exposes the gap that ordinary sparsity leaves behind. Each sub-decision is easy to find on its own: whether a column carries, or which digit it emits, maps to a small clean set of channels you can isolate as its own circuit. But these pieces never merge into one. Together they still fall short of a single minimal viable circuit for addition, and alone each one means nothing. Keep only the most strongly attributed mask and isolate it, and the model reproduces just 29.00% of full answers. Pushing recovery back near the full score takes roughly 90% of the MLP. The capability is there; the model’s own representation will not give it up cheaply.

The obstacle is the representation itself, not the way we rank its channels. For all but the simplest sub-decisions, the computation spreads across many channels, and features sit in superposition, so a single channel carries several behaviors at once [Elhage et al., 2022, Bricken et al., 2023]. A sharper pruning score cannot fix that, because the capability is not laid out along the channels the score reads. Sparse autoencoders chase a sparser basis by training a new one after the fact [Cunningham et al., 2023], and weight-sparse training builds the sparsity in from the start [Gao et al., 2025], whereas Prism instead reshapes the model’s own basis.

Collimation makes that change by fine-tuning the model under a behavior-preserving constraint, holding the input–output mapping fixed while pulling the target capability into a smaller set of channels. Attribution and masking then succeed where they failed on the base model. On two-digit addition, collimation lifts causal recovery from 29.00% to 91.33% while keeping roughly 5% of MLP channels and leaving task accuracy untouched. Appendix A traces the path to that result that the local recovery of sub-decisions, their failure to compose, and the identification of the basis as the operative assumption.

The focus of this paper is the collimation techniques we propose and the different ways they can be applied. Timing is one such setting where the adapter can train *before* channel selection, where it is still free to change which channels carry the capability so attribution can return a new and smaller mask, or *after* selection, where the mask is fixed and it can only reroute behavior through the channels already kept. We run both and report what each setting buys across integer arithmetic and function calling. We make the following contributions.

Contributions

1. **An extraction contract.** A substrate counts only if the channels we keep still carry the behavior after the rest are switched off. Finding channels tied to a behavior is not the same as showing the behavior runs through them (§2).
2. **A collimation frontier.** Collimation before attribution can change which channels carry the capability; collimation after attribution trains under a frozen mask and cannot change the surviving channel set. This axis explains why arithmetic becomes smaller while function calling packs behavior into a fixed substrate (§3,§A.4).
3. **Arithmetic becomes extractable after global collimation.** A pre-attribution rank-32, KL-constrained LoRA adapter moves a sparse MLP mask from 29.00% to 91.33% causal recovery while slightly reducing the channel count. (§5.1,§A.1).
4. **Function calling improves at a fixed substrate.** With Qwen3-8B masks held fixed, collimation raises BFCL recovery from 19.1% for the raw mask to 84.6–85.5%. The gain comes from the objective, not from a new mask, and the remaining errors split into schema/default completion versus value grounding (§5.2,§A.6).
5. **The representation changes without becoming lower-rank.** Pre-attribution collimation raises mean effective rank while concentrating task-relevant MLP activation energy inside the recoverable mask ($0.329 \rightarrow 0.462$). The model changes shape; it does not simply compress (§A.2).
6. **Intervention contracts decide what a circuit claim means.** Rescue keeps the original masked region intact and wraps it with a small adapter; replacement overwrites the region with a low-rank

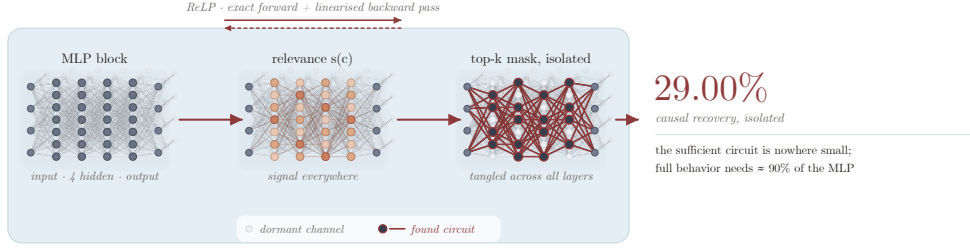


Figure 1: **Attribution in the native basis.** ReLP runs the exact forward pass and a single linearised backward pass, scoring every MLP intermediate channel by its relevance $s(c)$ to the teacher-forced log-probability of the target. Ranking by $s(c)$ and keeping the top- k defines a candidate mask m_k ; the complement is deactivated and the circuit is run in isolation. Because the capability is held in *superposition*, the isolated mask is *not* causally sufficient—two-digit addition recovers only 29.00%, and restoring full behavior needs $\approx 90\%$ of the MLP. Attribution finds the signal everywhere and the sufficient circuit nowhere small.

student. At the same EN $\rightarrow$ PT substrate, rescue and replacement give sharply different recoveries, so the contract must be pinned before circuit claims are compared (§5.3).

2 The Extraction Contract

The extraction test Every experiment in the paper has the same shape. First, score MLP channels for a target behavior. Second, keep the top-scoring channels and deactivate the rest under the task’s intervention contract. Third, measure how much of the original behavior remains. We call that fraction *recovery*. The rest of this section defines the objects used in that test.

Units of attribution Let M be a decoder-only transformer. We attribute over its MLP intermediate channels, the inputs to each block’s down-projection. These channels are the paper’s unit of extraction. They are also the native-basis axes tested by the sparse-neuron-basis premise. Formally, channels are indexed $c \in \mathcal{C}$ with $|\mathcal{C}| = L \cdot d_{\text{ff}}$ for L layers of width d_{ff} (for Qwen3-8B [Yang et al., 2025], $36 \times 12,288 = 442,368$ channels).

Attribution (ReLP) We score channels with relevance propagation (ReLP), the linearised-backward-pass attribution of Arora et al. [2026], whose title—*Language Model Circuits Are Sparse in the Neuron Basis*—is precisely the premise this paper tests. Attribution is taken with respect to a scalar metric: the teacher-forced log-probability the model assigns to the gold target continuation y given the prompt x ,

$$\mathcal{M}(x, y) = \sum_t \log p_M(y_t \mid x, y_{<t}). \quad (1)$$

ReLP scores target-relevant MLP channels. ReLP can be understood as a way to ask which MLP channels contributed to the log-probability of the target continuation? It runs the ordinary forward pass, then a single locally linear backward pass that propagates the scalar $\mathcal{M}$ to a relevance value for every channel. For each example we sum absolute relevance over continuation positions and average over an attribution set, yielding one score $s(c)$ per channel. Because the forward pass is unchanged, ReLP is single-pass and does not require ablating channels individually.

From scores to a mask Ranking channels by $s(c)$ and taking the top k defines a mask $m_k \subseteq \mathcal{C}$, the candidate circuit at budget k . A mask is tested by *deactivating the complement* $\mathcal{C} \setminus m_k$. The deactivation operator is part of the contract, and the three domains use three different operators:

- **Counterfactual patching (arithmetic).** Replace complement activations with those from a matched counterfactual input—the operator used to test 2-digit-addition sufficiency.
- **Mean-ablation (translation).** Replace complement activations with the dataset-mean activation.

- **Zero-isolation (function calling).** Set every unkept MLP intermediate channel to zero before the down-projection. The selected channels run with no constant offset from the rest. Every reported function-calling masked score uses this operator.

The choice of operator is crucial as the way you disable the unused channels determines what causal experiment you are actually running. Function-calling *training* masks initially used mean-ablation while *evaluation* always used zero-isolation; this train/eval mismatch is itself a finding (§B.5, §5.2), and aligning the training mask to zero-isolation is a principal driver of the best result.

Sufficiency and recovery A mask is *causally sufficient* to the extent the masked model reproduces the unmasked behavior. We report **recovery**

$$R(m_k, \theta) = \frac{\text{score}(M_\theta \text{ under } m_k)}{\text{score}(M \text{ unmasked})}, \quad (2)$$

where θ is an optional collimation adapter. For arithmetic, “score” is exact full-answer accuracy; for translation, XCOMET-XXL; for function calling, (normalized) exact structured match.

Collimation A collimator is an adapter θ , trained with a *behavior-preserving* objective and then evaluated under a mask. In optics, a prism resolves a beam into its spectrum only once that beam has been collimated. The adapter plays the same role here, concentrating a capability into a coherent set of channels so attribution can isolate a sufficient substrate from it. It is worth stating precisely what collimation is and is not, because superficially similar operations do something different:

Here, *behavior-preserving* names the training contract i.e. the collimator is trained to preserve the target behavior or policy while changing where that behavior is carried internally. The preserved behavior is domain-specific—arithmetic answers, translation quality, or tool-call policy—and the substrate is judged only under the stated intervention contract. This is why isolation is a causal claim about a channel substrate inside a fixed scaffold, not a claim that the adapter is a standalone compressed model.

- It is **not ordinary fine-tuning**. Fine-tuning changes the model’s input–output map to improve a metric. Collimation is constrained to *hold that map fixed*—task accuracy and, where applicable, a KL leash to the base distribution are kept constant—so the adapter cannot buy recovery by becoming a better model. Any change it makes is purely internal.
- It is **not pruning or weight rearrangement**. Collimation does not select, reorder, or delete existing weights, and it does not distill into a smaller network. The model keeps its full size and parameters; the adapter adds a small behavior-preserving perturbation whose only job is to move the *representation*—to change which directions carry the capability.
- It is therefore a **basis-change operator**, not a capacity operator. The goal is to relocate an existing capability into channels the standard mask can keep, not to add, remove, or improve capability. Empirically (§A.2) it does so by concentrating task-relevant activation energy into the mask while leaving behavior unchanged, and it can even *raise* the model’s effective rank—the opposite of compression.

The central object of this paper is the map from *collimation objective* to recovery at a fixed m_k ; one thing it depends on, when collimation runs relative to the mask, is the subject of §3.

Intervention contracts We distinguish two contracts at the same substrate: **rescue** keeps the masked region’s original weights intact and wraps it with a small adapter (the dense base remains in place); **replacement** overwrites the region with a standalone low-rank module (the region’s contribution comes entirely from the student). These are different objects and, as §5.3 shows, give very different recoveries.

3 The Collimation Frontier

Both regimes “collimate” a model so a capability becomes extractable, but they are *not the same operation*. The simple version is this - collimation before channel selection can change which channels matter; collimation after channel selection cannot. The formal variable is **when collimation**

Table 1: Two regimes of collimation.

	Collimate before choosing channels	Collimate after choosing channels
Formal name	Pre-attribution collimation	Post-attribution collimation
Mask during training	absent; trains on the full model	present; complement is already deactivated
Order	collimate $\rightarrow$ attribute $\rightarrow$ mask	attribute $\rightarrow$ mask $\rightarrow$ collimate
What can change	which channels carry the capability	how behavior routes through fixed channels
Re-attribution result	can find a smaller sufficient mask	returns essentially the same mask (IoU 0.92)
Frontier effect	moves left; circuit shrinks	fixed; intra-budget packing

happens relative to attribution, equivalently **whether the mask is present while the collimator trains**. This axis, not the particular distillation loss, determines whether collimation can shrink a circuit.

3.1 Two regimes

Pre-attribution (global) collimation The collimator trains on the *full, unmasked* model and is merged into the weights; attribution and masking are run *afterward*, on the collimated model. Because no mask exists during training, the adapter’s gradients see the whole network and it is free to reshape the global representation. The mask is therefore an *output* of collimation i.e. re-attribution on the collimated model can return a *different, smaller* sufficient set. Order: **collimate** $\rightarrow$ **re-attribute** $\rightarrow$ **mask**. This is the arithmetic collimator (§B.1; Fig. 5).

Post-attribution (masked) collimation Attribution fixes a mask on the *base* model; the complement is deactivated; the collimator then trains *under that lesion*, so gradients flow only through the surviving channels. The mask is an *input* to collimation and is never recomputed to shrink it—by construction the adapter cannot change *which* channels survive, only *how well* behavior is routed through them. Order: **attribute** $\rightarrow$ **mask** $\rightarrow$ **collimate**. This is the translation rescue and the entire function-calling family (§B.2–§B.5; Fig. 6).

These regimes answer different questions. Pre-attribution collimation asks *can we reshape the model so a smaller circuit becomes sufficient?* Post-attribution collimation asks *given this circuit, how much behavior can we route through it?* The “intra-budget packer” result (§A.4)—that collimation fortifies and shrinks the substrate—is a property of the **post-attribution** regime specifically, a direct consequence of freezing the mask before the adapter runs.

4 Experimental Setup

Table 2: Capabilities, models, and base scores.

Capability	Model	Eval data	Metric	Base score
2-digit addition	Qwen2.5-Math-1.5B	exhaustive 2-digit sums, held-out	exact full-answer	94.86%
EN $\rightarrow$ PT translation	HY-MT1.5-1.8B	NTREX-128 (held out)	XCOMET-XXL	0.9283
Function calling	Qwen3-8B	BFCL v3 single-turn/call (1007 rows)	(norm.) exact match	64.05% / 65.94%

The model families are Qwen2.5-Math [Yang et al., 2024], Qwen2.5 [Qwen Team, 2025], Qwen3 [Yang et al., 2025], and HY-MT1.5 [Zheng et al., 2025]. Translation uses NTREX-128 as its held-out evaluation set [Federmann et al., 2022].

Function-calling recovery uses two base counts. The raw exact score divides by 645, the stricter normalized score by 664. Recovery always means masked score divided by the corresponding unmasked base score. The normalized scorer is stricter than the official BFCL scorer an omitted optional or

default argument counts as wrong. The 1007-row slice is filtered to single-turn, single-function, one-ground-truth-call (composition by source: simple 400, live_simple 258, exec_simple 100, java 100, javascript 50, sql 99). Attribution objective is teacher-forced full gold continuation log-probability throughout.

5 Results

5.1 Arithmetic: pre-attribution collimation moves the sparsity frontier

The base model loses exactly where a circuit claim should win. Sparse masks find arithmetic *signal* but not a sufficient *circuit*: the best compact mask recovers 29.00% of full answers, and 99.53% recovery requires retaining 90.61% of MLP channels. After *pre-attribution* rank-32 KL-LoRA collimation—adapter trained on the full unmasked model, merged, then re-attributed, with task accuracy held at the base 94.86%—a *smaller, differently-located* mask becomes sufficient. The comparison in Figure 2 is not “same mask, more recovery”; it is “the mask we find after reshaping the model is smaller *and* recovers 3 $\times$ more.” That is only possible because attribution runs on the collimated model.

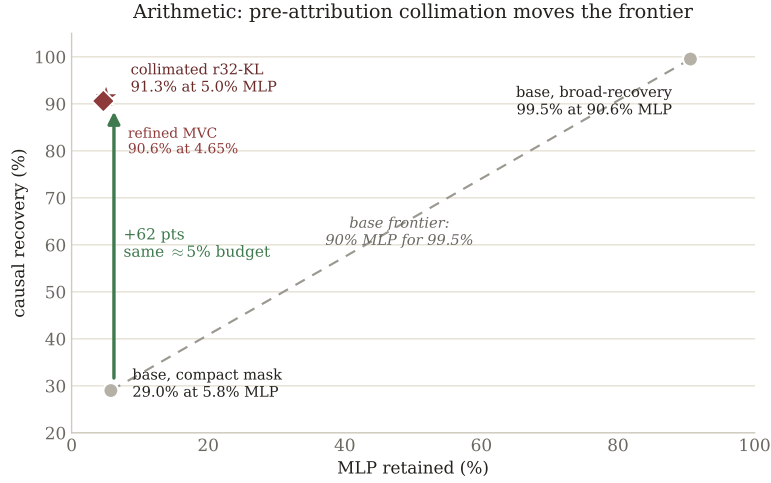


Figure 2: Sparsity–recovery frontier, before and after collimation (Qwen2.5-Math-1.5B, 2-digit addition). On the base model a compact mask (14,436 ch., 5.75% MLP) reaches only 29.0%, and matching 99.5% recovery needs $\approx 90\%$ of the MLP. Pre-attribution rank-32 KL-LoRA collimation yields a *smaller* sufficient mask (12,661 ch., 5.05% MLP; refined minimal viable circuit, MVC, 11,672 ch., 4.65%) at 91.3% recovery—a +62-point gain at the same $\approx 5\%$ budget.

The refined MVC drops layers {5, 9, 12, 13, 14, 17, 27}, reaching 90.60% recovery at 11,672 channels (4.65% of the MLP universe, $\approx 53.8\text{M}$ selected scalar weights, $\approx 3.49\%$ total-model-equivalent parameters). This is a behavioral-recovery claim under a sparse causal mask i.e. the masked network still uses embeddings, attention, normalization, residual routing, and the intervention scaffold but is sparsely deployable.

5.2 Function calling: searching post-attribution collimators at a fixed substrate

Function calling post-attribution collimation is closer to reconstruction of capability of the model in a smaller substrate. Attribution alone yields no small substrate (Figure 3): the top-18% of MLP is dead, and recovery reaches only 61.4% at 54% of MLP. Unlike §5.1, every collimator here is *post-attribution*: we fix two base-model substrates ($k=160\text{k} = 36.2\%$ MLP, $k=240\text{k} = 54.3\%$ MLP), **isolate** them by zeroing the complement at the MLP down-projection input (§2; the selected channels run with no contribution from the rest, not against a mean baseline), and search collimation *objectives* under the frozen mask. Because the mask never moves (§A.4), recovery in Figure 3 isolates the effect of the objective alone—the channels are identical within each block. The sampled-token $\rightarrow$ hybrid

jump (43.1% $\rightarrow$ 84.6%) is partly an objective change and partly the train/eval masking alignment of §B.5.

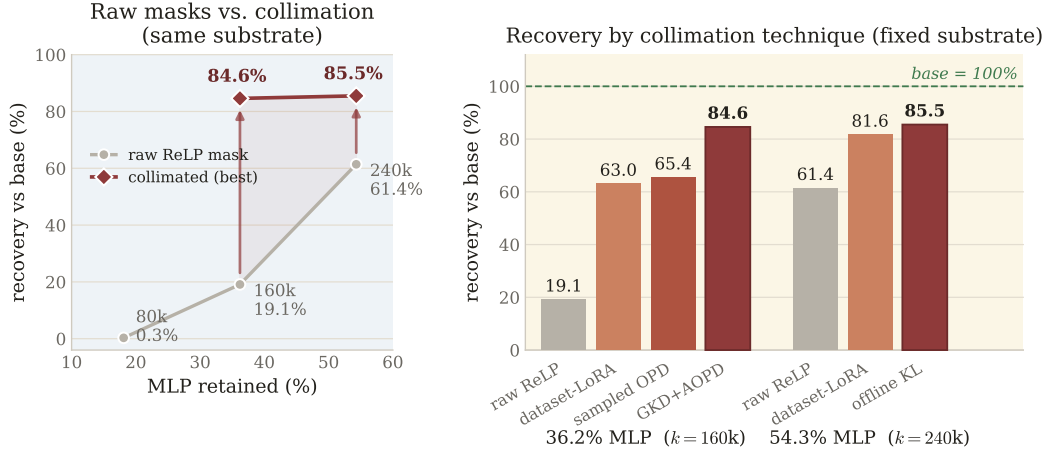


Figure 3: Function calling (Qwen3-8B, BFCL). **Left:** raw ReLP masks recover little without collimation (0.3/19.1/61.4% at 18.1/36.2/54.3% MLP); at the two fixed substrates our best post-attribution collimator lifts recovery to 84.6% and 85.5% on the *same* channel budget. **Right:** recovery by collimation technique at the two fixed substrates ($k=160k = 36.2\%$ MLP, $k=240k = 54.3\%$ MLP); normalized exact, recovery vs base 664. The channels are identical within each block—only the training objective changes; the best collimator per substrate is shown in maroon.

First, **distilling the base policy beats fitting dataset labels:** offline policy-KL gives 568/1007 (85.5%) at $k=240k$, +24 correct over the dataset collimator, and adding CE to labels *hurts* (−16). Second, **a constrained on-policy hybrid nearly closes the substrate gap:** at $k=160k$ the GKD+AOPD hybrid (generalized knowledge distillation plus anchored on-policy distillation; §B.5) reaches 562/1007 (84.6%), +140 correct over the supervised collimator (422) and within one point of the larger $k=240k$ circuit. The channels are identical across each block of Figure 3; only the training objective changes.

Naive on-policy distillation is unstable: pure prefix-OPD collapses (36/1007 from a student rollout; 0/1007 from a base rollout) because early masked rollouts visit degenerate states. The sampled-token variant avoids collapse and fixes schema/default completion (+12) but introduces value corruption (e.g. 9.8 $\rightarrow$ 99.0, AAPL $\rightarrow$ Apple Inc.). The hybrid’s positive-only online term and offline anchor remove this failure mode.

5.3 Translation: rescue versus replacement at the same substrate

On EN $\rightarrow$ PT (HY-MT1.5-1.8B), the masked-KL *rescue* collimator recovers strongly: at $k=160k$ ($\approx 81\%$ of MLP) a rank-8 LoRA reaches XCOMET-XXL 0.8121, i.e. 87.5% of the unmasked teacher (0.9283). The rank floor is gentle (rank-32 optimal at $k=120k$; rank-64 buys drift not recovery). The substrate is stable under collimation (channel-level intersection-over-union (IoU) 0.92 between base- and collimated-model masks at $k=160k$).

The *replacement* contract behaves differently at the same substrate. A region selected by automated circuit discovery (ACDC; Conmy et al., 2023) and overwritten by a per-layer low-rank student recovers only XCOMET 0.1736 (rank-128, 18.7% of teacher)—yet the *same region under the original activations* recovers 0.7544 (81.3%). The region is intact; the low-rank student simply fails to reproduce its channel-level activations from output-space supervision. **The $4\times$ rescue/replacement gap is a contract effect, not a substrate effect**, and we argue it underlies many cross-study disagreements about whether a “circuit” exists.

6 Controls, Scope, and Prior Work

Matching task accuracy is easy with any fine-tuning or distillation process but making arithmetic recover at 5% of MLP channels is specific to rank-32 KL-constrained pre-attribution collimation.

No-KL LoRA, lower-rank KL-LoRA, and full-parameter SFT all reach the task but need roughly $14\times$ more channels for the same masked recovery, while random same-size masks recover at most 2.2%. Collimation also does not simply compress the hidden state: mean effective rank rises, while task-relevant MLP activation energy concentrates inside the recoverable mask. The control table and representation diagnostics are in Appendix A.1–A.2.

The post-attribution result has the opposite contract. Once the mask is frozen, no adapter can change which channels survive. Chunk-drop tests, k240-to-k160 transfer, and re-attribution after rescue all show the same fact: the adapter fortifies the substrate (Appendix A.4). The two regimes are separated by collimation happening pre or post attribution. Pre-attribution collimation can move the frontier; post-attribution collimation can pack behavior inside it.

Prism sits at the intersection of sparsity, distillation, and attribution, but the object is different from each. Pruning and sparse training reduce model-level capacity [Louizos et al., 2018]; distillation trains a student against a teacher [Hinton et al., 2015, Kim and Rush, 2016]; LoRA supplies the low-rank adaptation mechanism [Hu et al., 2022]; attribution and circuit-discovery methods score components whose interventions affect behavior [Vig et al., 2020, Wang et al., 2023, Goldowsky-Dill et al., 2023, Conmy et al., 2023]. Prism uses these ingredients to ask a sharper efficiency question: can a named capability still run through a specified sparse MLP-channel substrate after the complement is disabled?

Limitations and impact We report function calling only under post-attribution collimation; the pre-attribution setting on that task is untested here. The upside is capability-level efficiency and sharper intervention tests; the risk is that easier extraction also makes narrow behaviors easier to isolate and reuse, so any public artifact should ship with its intervention contract, scorer, intended-use boundary, and failure modes, not only a mask file. This needs to be applied across various domains and also scenarios where the correct answers can be a wide range of options.

7 Conclusion

We began with the ordinary efficiency question, how much of a model can be removed before it stops working, and replaced it with a sharper one. The question is not how small the model can be, but how small a single capability can live while the rest of the network is switched off and the behavior still runs. That reframing is the whole paper, because a capability is not efficient when attribution merely says its channels look important; it is efficient only if it survives the lesion. Collimation is how we make a model give one up—a behavior-preserving step that moves where a capability is carried without changing what the model says. What that step can buy depends on how much freedom it has when it runs. Given the whole network, it relocates a capability into a smaller substrate: arithmetic recovery climbs from 29.00% to 91.33% through roughly 5% of the MLP on a 1.5B math model. Given a substrate already fixed, it instead packs more behavior through those same channels and the training objective becomes the whole contest: function-calling recovery rises from 19.1% to 84.6% at a fixed budget on an 8B model.

What we have shown is a first carving, two capabilities reshaped with a single knob. The immediate open question is whether pre-attribution collimation behaves on a function-calling model as it did on arithmetic—a setting we have not yet run. The frontier we most want to reach next is agentic behavior, the multi-step planning and tool use that a single function call only begins, and we mean to extract that frontier next. Past that point lies the larger aim, since collimation acts on MLP channels today while the same question reaches into attention, into activation sparsity, and into how all of these compose with quantization. The destination is a model you deploy for the one behavior you actually use, served at a fraction of the cost, while the capabilities worth extracting keep arriving every few months in open weights. Sparsity has always been something we discover only after a model is trained, and we would like it to become something we can ask for, one capability at a time.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. *ICLR*, 2024.

- Aryaman Arora, Zhengxuan Wu, Jacob Steinhardt, and Sarah Schwettmann. Language model circuits are sparse in the neuron basis. *arXiv:2601.22594*, 2026.
- Trenton Bricken et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.
- Arthur Conmy, Augustine Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. *NeurIPS*, 2023.
- Nelson Elhage et al. Toy models of superposition. *Transformer Circuits Thread*, 2022.
- Nuno M. Guerreiro, Ricardo Rei, Daan van Stigt, Luisa Coheur, Pierre Colombo, and André F. T. Martins. xCOMET: Transparent machine translation evaluation through fine-grained error detection. *Transactions of the Association for Computational Linguistics*, 12:979–995, 2024.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.
- Ricardo Rei, Craig Stewart, Ana C. Farinha, and Alon Lavie. COMET: A neural framework for machine translation evaluation. *EMNLP*, 2020.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. *ICML*, 2025.
- Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. Investigating gender bias in language models using causal mediation analysis. *NeurIPS*, 2020.
- Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: A circuit for indirect object identification in GPT-2 small. *ICLR*, 2023.
- Nicholas Goldowsky-Dill, Chris MacLeod, Lucas Sato, and Aryaman Arora. Localizing model behavior with path patching. *arXiv:2304.05969*, 2023.
- Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv:2309.08600*, 2023.
- Leo Gao et al. Weight-sparse transformers have interpretable circuits. *arXiv:2511.13653*, 2025.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.
- Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. *EMNLP*, 2016.
- An Yang et al. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv:2409.12122*, 2024.
- Qwen Team. Qwen2.5 technical report. *arXiv:2412.15115*, 2025.
- An Yang et al. Qwen3 technical report. *arXiv:2505.09388*, 2025.
- Mao Zheng, Zheng Li, Tao Chen, Mingyang Song, and Di Wang. HY-MT1.5 technical report. *arXiv:2512.24092*, 2025.
- Christian Federmann, Tom Kocmi, and Ying Xin. NTREX-128: News test references for MT evaluation of 128 languages. *SUMEval*, 2022.
- Chen Yang, Ran Le, Yun Xing, Zhenwei An, Zongchao Chen, Wayne Xin Zhao, Yang Song, and Tao Zhang. ToolMind technical report: A large-scale, reasoning-enhanced tool-use dataset. *arXiv:2511.15718*, 2025.
- Zuxin Liu et al. APIGen: Automated pipeline for generating verifiable and diverse function-calling datasets. *NeurIPS Datasets and Benchmarks Track*, 2024.
- Weiwen Liu et al. ToolACE: Winning the points of LLM function calling. *ICLR*, 2025.
- Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. *ICML*, 2019.
- Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. *EUSIPCO*, 2007.
- Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.
- Ganqu Cui et al. Process reinforcement through implicit rewards. *arXiv:2502.01456*, 2025.

NLLB Team et al. No language left behind: Scaling human-centered machine translation. *arXiv:2207.04672*, 2022.

Maja Popović. chrF++: Words helping character n-grams. *WMT*, 2017.

Christos Louizos, Max Welling, and Diederik P. Kingma. Learning sparse neural networks through L0 regularization. *ICLR*, 2018.

A Results and their significance

A.1 The arithmetic effect requires KL-constrained rank-32 collimation

The arithmetic result is not generic “fine-tune-then-prune.” That escape hatch dies under controls.

Table 3: Arithmetic collimation controls (channels required for $\sim 90\%$ recovery).

Variant	Outcome
rank-32 KL (ours)	91.33% autoregressive recovery at 12,661 channels
rank-32 no-KL LoRA	49.53% at compact scale; $\sim 167,065$ channels for 89.80%
rank-8 KL LoRA (best lower rank)	90.47% at $\sim 170,259$ channels
full-parameter SFT	90.80% at $\sim 179,311$ channels
wrong-task LoRA controls	no sparse arithmetic mask appears
random same-size mask (12,661 ch.)	1.93–2.20% recovery

Matching task accuracy is achievable many ways; making the capability *extractable at 5% of channels* is specific to KL-constrained rank-32 collimation. Lower rank, no KL, and full SFT all require $\sim 14\times$ more channels for the same recovery. Cross-model: on the non-math Qwen2.5-1.5B, rank-32 KL reaches 90.07% at 114,026 channels versus 157,696 (no-KL)—the effect persists but pretraining matters (the math-tuned model sits closer to a sparse solution).

A.2 A representational account

The intuitive hypothesis—collimation lowers hidden-state rank—is false. Mean effective rank *rises* (base 9.878 $\rightarrow$ r32-KL 11.789; no-KL 10.183). Instead, collimation (i) moves representations away from the base (lower centered kernel alignment, CKA, and top-10 subspace overlap) and (ii) concentrates task-relevant MLP activation energy inside the mask: the mask’s share of MLP activation energy rises from 0.329 (base) to 0.462 (collimated). Collimation reshapes and *concentrates* the representation into a mask-recoverable subset rather than compressing it dimensionally.

A.3 Specificity and brittleness

The collimated mask is a specific channel set, not a generic large one. Random same-size masks recover $\leq 2.2\%$ (Table 3). The mask is also brittle in the diagnostic sense: dropping 5% of selected channels lowers recovery to 61.20%, dropping 10% to 31.53%. The sparse rank-32 KL mask does not transfer to the base or no-KL models—sufficiency is a property of the collimated representation, not a portable channel list.

A.4 Post-attribution collimation cannot shrink the substrate (intra-budget packing)

The claim is scoped: a collimator trained *under a frozen mask* fortifies whatever substrate it is given but does not discover a smaller one. This is a structural consequence of the protocol, not of the capability. The mask is an input the adapter’s gradients cannot edit (§3.1). That is the contrast with pre-attribution arithmetic (§5.1), where collimation ran before attribution and the frontier *did* move (14,436 $\rightarrow$ 12,661 channels at far higher recovery).

Three diagnostics establish the packing property within the post-attribution regime.

1. **Chunk-drop damage does not compose.** On a collimated $k=240k$ model, the first 10k ReLP channels are a hard anchor: removing them drops the score to 0/1007. Four later chunks (030–040k, 040–050k, 100–110k, 160–170k) have *negative* single-chunk damage, meaning each removed alone slightly raises the score. If those chunks were independently disposable, removing all four together should help. It does the opposite: the resulting 200k-channel circuit drops to 42.2%, well below the full 54.0%. They are conditionally useful inside the full substrate, not a smaller core waiting to be carved out.
2. **The larger repair does not fold into the smaller mask.** A $k=240k$ repair evaluated on its own $k=160k$ subset reaches only 369/1007, below a fresh $k=160k$ run, and a gentle re-collimator makes it worse. The larger repaired substrate has not secretly learned a compact inner substrate.

3. **Re-attribution returns the same masked substrate.** On translation, re-attributing after masked-rescue collimation selects essentially the same channels (IoU 0.92) and does no better. The collimator was trained under the mask, so it never had access to the global representation needed to move the frontier.

Consequently, at a fixed post-attribution substrate the collimation *objective* is the only effective lever (§5.2). The untested cell of the design is the obvious next experiment: apply the *pre-attribution* arithmetic protocol—global KL-collimation of the full model, then re-attribute—to function calling. Crucially, the “intra-budget packer” result is specific to the frozen-mask regime and does **not** bound what pre-attribution collimation might do on this task.

A.5 The rank floor

Across function-calling collimators, rank obeys $r=16 \succeq r=32 \succeq r=64$ (e.g. offline policy-KL at $k=240k$: r8 546, r16 568, r32 533). Additional rank buys representational drift, not recovery—consistent with the translation rank floor (rank-64 reduces unmasked quality). Cold-start hybrids scale monotonically with rank (fresh r8 272 < r16 370 < r64 495) but even fresh r64 stays 58 correct below the supervised-init r32 hybrid: the collimation *curriculum*, not raw capacity, carries the gain.

A.6 Failure-mode decomposition (function calling)

Structured failure buckets explain what each collimator isolates. Going from raw mask to the dataset collimator (v2 r16, $k=240k$) fixes *structure*: exact tool/arg naming (−94, syntax −31, while schema/default completion worsens (+36) and value grounding (−51) remains the largest residual. On-policy signal then trades buckets: it improves schema completion but degrades value grounding; a ToolACE exact-value repair [Liu et al., 2025] reverses the trade (562 → 561, a wash). No single objective wins both grounding and schema completion—the open frontier (§H).

Arithmetic failures are likewise structured: recovery is highest on single-carry (97.65%) and no-carry (93.83%) regimes and lowest on cascading multi-carry (carry_2_3 88.62%), localizing where the missing computation lives.

A.7 Sampling uncertainty does not explain the collimation gains

Function-calling exact match is a binomial outcome over the fixed 1007-row BFCL slice, so a Wilson score interval on each masked proportion bounds the evaluation-set sampling uncertainty (Figure 4). At the 36.2% substrate the raw mask scores 12.2% (95% CI 10.3–14.4) and the GKD+AOPD hybrid reaches 55.8% (52.7–58.8); the gap spans many interval widths, so the gain is not an artifact of the evaluation sample. The same intervals bound what the data cannot separate. Dataset-LoRA (41.5%, 38.5–44.6) and sampled-token OPD (43.1%, 40.1–46.2) overlap, so that one-step ordering sits inside sampling noise even though the raw-to-best gap does not. These intervals cover sampling uncertainty on the fixed slice only. They do not capture training-seed variance, which would require repeated independent collimation runs (§H).

A.8 Extractability depends on the model: the slack hypothesis

The substrate one can reach is not purely a property of the capability or the collimator; it is also a property of the *model* the capability lives in. The clearest contrast is between the math-tuned arithmetic model and the translation model. Qwen2.5-Math-1.5B admits a ~5%-of-MLP arithmetic circuit after collimation, and the cross-model control sharpens the point: the non-math Qwen2.5-1.5B needs 114,026 channels for roughly 90% recovery, versus 11,672 channels for the math model. A model whose pretraining already organized the capability cleanly is *closer to a sparse solution* and easier to extract. HY-MT1.5-1.8B, by contrast, is a heavily **distilled** translation specialist [Zheng et al., 2025], and its EN→PT substrate never became small—the best rescue still runs through ~80% of the MLP, and collimation could not move that frontier.

Our working hypothesis is that heavy distillation spends the slack collimation needs. A model distilled hard onto a single task is trained to reproduce a teacher’s outputs across that task distribution with little unused representational freedom; the capability may already be packed densely and redundantly across the network, leaving fewer clean directions for a collimator to rotate into an extractable

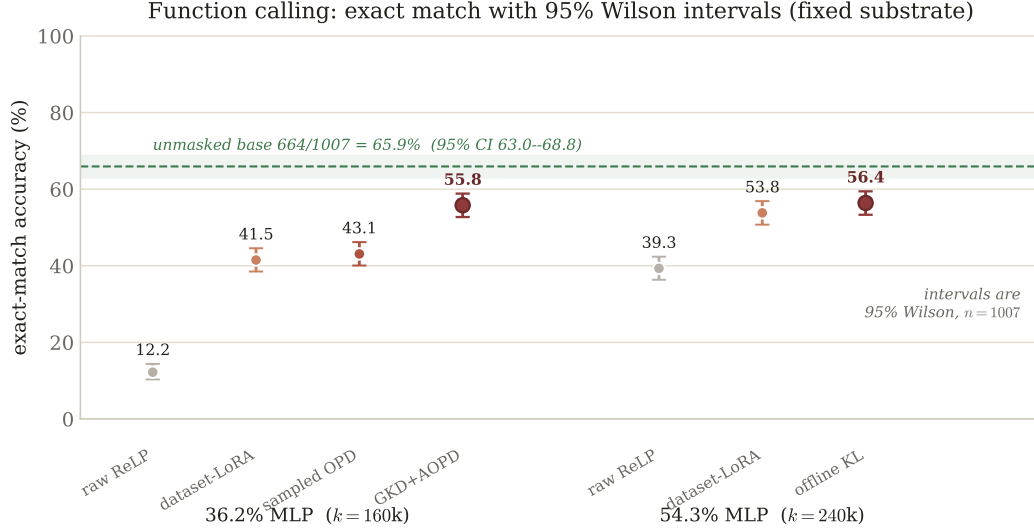


Figure 4: **Function-calling exact match with 95% Wilson confidence intervals** ($n=1007$). Each point is the masked exact-match accuracy at a fixed substrate; error bars are Wilson score intervals for the binomial proportion, computed from the raw correct counts. The dashed line and shaded band are the unmasked base ($664/1007 = 65.9\%$) with its own interval. The collimation lift (raw $\rightarrow$ best) exceeds many interval widths at both substrates, while neighbouring collimators (dataset-LoRA vs. sampled OPD) overlap. The intervals capture evaluation-set sampling uncertainty on the fixed slice, not training-seed variance.

substrate. Under this hypothesis, the contrast is not “task-tuned models help or hurt” in general. Light specialization can organize a capability and make it easier to extract, as in the math model; aggressive distillation can consume the slack that extraction needs, as HY-MT appears to do. The practical reading is sharp but conditional: collimation relocates a capability into representational room the model still has, and models with more room are better extraction substrates.

B Collimation Objectives

The objective details now have a job: explain how the results above were produced, not carry the reader’s first understanding of the paper. The sequence is controlled. First, use labels and a teacher to rescue behavior through a fixed mask. Second, drop labels and distill the base policy because the teacher’s uncertainty is useful signal. Third, let the masked student visit its own states, which helps schema completion but corrupts values. Fourth, keep the online benefit while adding back the value-preserving policy anchor.

B.1 Pre-attribution KL-LoRA collimation

Algorithm Freeze the base weights M . Train a rank- r LoRA adapter θ on the *full, unmasked* model with

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \beta \text{KL}(p_M(\cdot | x) \| p_{M_\theta}(\cdot | x)), \quad \beta = 0.05. \quad (3)$$

Sweep the merge scale α (the LoRA delta is added as $W + \alpha \Delta W$), discard any α that moves task accuracy off the base value, and merge at the *lowest-KL* surviving scale ($r=32$, $\alpha=0.55$). Run ReLP and top- k masking on the merged model.

Reading the objective The KL is written with the *base model as the reference distribution* p_M —a forward, mass-covering KL. This direction is the whole point. Mass-covering means the collimated model is penalized wherever it puts *less* probability than the base did; it cannot drop, sharpen, or re-rank any of the base’s output mass. Matching argmax accuracy would be a far weaker constraint (it permits arbitrary reshaping of the rest of the distribution); forward KL pins the entire predictive distribution token-for-token, so “behavior-preserving” holds at the level of the full output law, not

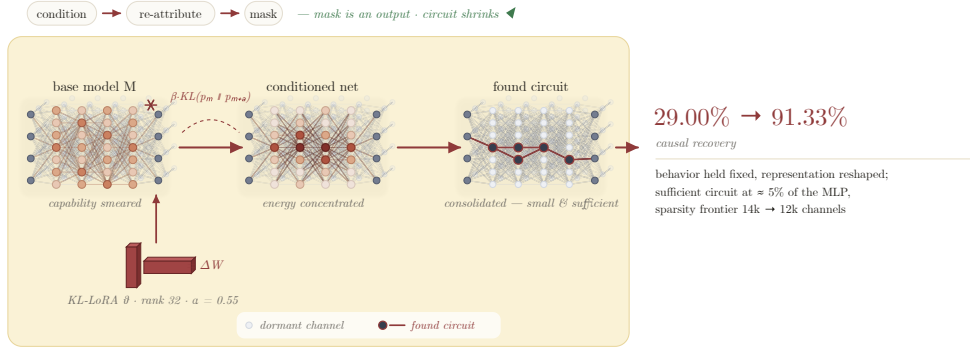


Figure 5: **Low-rank collimation (pre-attribution)**. A rank-32, KL-constrained LoRA adapter trains on the *full, unmasked* model under $\mathcal{L} = \mathcal{L}_{\text{task}} + \beta \text{KL}(p_M \parallel p_{M_\theta})$, then merges at the lowest-KL scale ($\alpha=0.55$). Behavior is held fixed while the representation is reshaped, *concentrating* task-relevant energy into a compact set of channels. Attribution is run *afterward*: the mask is an **output** of collimation. The sparsity frontier moves left—the sufficient circuit shrinks ($\approx 14\text{k} \rightarrow 12\text{k}$ channels) and causal recovery rises from 29.00% $\rightarrow$ 91.33% at $\approx 5\%$ of the MLP.

merely the answer. $\beta=0.05$ makes it a leash, not a clamp: large enough that the gradient cannot reduce KL by changing outputs, small enough that the adapter still has room to move internal activations.

Why this should—and does—reshape rather than relearn The constraint geometry is the mechanism. The task loss wants to express a capability; the KL forbids expressing it as any change in outputs; the only remaining degrees of freedom are *internal*. So the adapter discharges its gradient by rotating and concentrating the representation while the output law stays fixed. The merge-scale sweep then keeps the gentlest such rotation that preserves accuracy, so we are measuring the effect of a minimal reshaping, not of a different model. Three predictions follow, all borne out (§A.1–§A.2): (a) accuracy-matching alone should be easy and insufficient—confirmed, since no-KL LoRA, lower rank, and full SFT all reach base accuracy yet need $\sim 14\times$ more channels for the same masked recovery; (b) the reshaping should not look like dimensional compression—confirmed, mean effective rank [Roy and Vetterli, 2007] *rises* ($9.878 \rightarrow 11.789$) and CKA [Kornblith et al., 2019] to the base *drops*; (c) the capability’s energy should move *into* the eventual mask—confirmed, the mask’s share of MLP activation energy rises $0.329 \rightarrow 0.462$. Rank-32 is the floor where this is strong without drift (§A.5). Because attribution is recomputed on the merged model, this is the only collimator in the paper that can return a *new, smaller* sufficient set.

B.2 Post-attribution: masked-KL rescue (dataset-target)

Algorithm Fix a base-model mask m_k , deactivate its complement, and train a LoRA *under the mask* with three terms,

$$\mathcal{L} = \underbrace{\text{KL}(p_{M^{\text{full}}} \parallel p_{M_\theta^{\text{masked}}})}_{\text{(a) masked KL to teacher}} + \lambda_{\text{ce}} \underbrace{\text{CE}(y^*)}_{\text{(b) gold anchor}} + \lambda_u \underbrace{\text{KL}(p_{M^{\text{full}}} \parallel p_{M_\theta^{\text{full}}})}_{\text{(c) unmasked guardrail}}. \quad (4)$$

Term by term (a) The student forward pass is computed *with the complement deactivated*, the teacher *intact*; matching the lesioned student’s output to the intact teacher’s is precisely the statement “carry the behavior through the surviving channels,” and the mass-covering direction again forces the student to reproduce the teacher’s full spread over acceptable calls rather than collapsing to one. (b) The student also sees the gold target y^* at small weight, which matters where the teacher itself is uncertain or degenerate on hard rows—there the KL alone gives a flat, low-information target and the CE supplies a definite gradient. (c) The guardrail evaluates the collimated model with the mask *off* and keeps it close to base, which stops the adapter from “solving” the masked task by becoming a network that only works masked. The guardrail is what makes the unmasked collimated model remain a valid model: empirically the unmasked score never falls below $\sim 66\%$, at or above base, across every function-calling rescue.

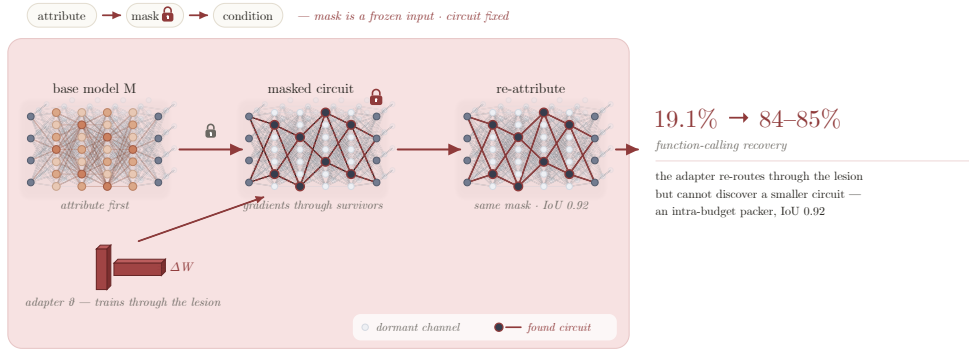


Figure 6: **Masked low-rank collimation (post-attribution)**. Attribution fixes the mask *first*; the complement is deactivated by zero-isolation and the adapter trains *through the lesion*, so gradients flow only along the surviving channels. The mask is a frozen **input** the adapter cannot edit—it can re-route behavior through the substrate but never discover a smaller one. This *intra-budget packer* fortifies the circuit (function calling 19.1% → 84–85%) yet leaves the frontier fixed: re-attribution returns essentially the same mask (IoU 0.92). Shared by the translation rescue (§B.2) and the entire function-calling family (§B.2–§B.5).

Why data, not just objective, decides this one Because the adapter is free to absorb whatever the training distribution emphasizes, the rescue is acutely data-sensitive. A rough general mix (repair-v1: ToolMind arg-heavy rows plus synthetic hotfixes) *helped the masked circuit* (+3.5 pts at $k=160k$) while *degrading the full model* (−4.6 pts unmasked)—the adapter shifted the model’s global behavior toward the mix instead of just rerouting the target capability, exactly the failure the guardrail is meant to bound and the mix overran. A strict, single-call, schema-validated mix (v2) avoided this and gave the best dataset-collimator numbers. This is the translation rescue collimator (mean-ablation throughout) and the first function-calling collimator; the function-calling instantiation trains under **mean-ablation** but is scored under **zero-isolation** (§2)—the mismatch §B.5 removes.

B.3 Post-attribution: offline masked policy distillation

Algorithm Drop the dataset CE and treat the *unmasked base* as a *policy teacher*. On gold prefixes $x_{<t}$, match full-vocabulary distributions over the tool-call tokens,

$$\mathcal{L} = \text{KL}\left(p_M(\cdot \mid x_{<t}) \parallel p_{M_\theta^{\text{masked}}}(\cdot \mid x_{<t})\right). \quad (5)$$

Why a distribution beats a label here The base model already solves the task at full capacity, so at each tool-call position its distribution is not a guess but a *calibrated* spread: it puts mass on the canonical argument *and* on the legitimate alternatives—optional fields, default values, equivalent formats—in roughly the proportions a correct policy should. The one-hot gold label throws all of that away and asserts a single token. Distilling the distribution therefore transfers strictly more of the capability than CE on labels can. The empirical test is decisive and, at first sight, counter-intuitive: adding even a small CE term *lowers* recovery (568 → 528 at $k=240k$, −40 correct). The reason is that on tool calls many positions are genuinely multi-valued; CE punishes the student for hedging the way the teacher *correctly* hedges, so it drags the student off the teacher’s calibrated distribution toward an over-confident single token. Pure policy-KL is best precisely because the teacher’s uncertainty is signal, not noise. The limitation is structural: gold prefixes only ever show the student states the *teacher* visits, never the lesion-distorted states the masked student will actually encounter at inference—which is what §B.4 attacks.

B.4 Post-attribution: on-policy distillation (OPD) and the sampled-token variant

Algorithm Let the masked student roll out its own continuations and have the teacher score the *student-visited* states. The naive prefix variant distills on the full visited distribution. The sampled-token, Prime-style variant [Cui et al., 2025] computes, for each sampled token y_t , a detached

advantage and a PPO-style importance ratio [Schulman et al., 2017] against the rollout policy π_{roll} (the student at sampling time):

$$a_t = \log p_{\text{teacher}}(y_t) - \log p_{\text{roll}}(y_t), \quad \rho_t = \exp(\log p_{\theta}(y_t) - \log p_{\text{roll}}(y_t)), \quad (6)$$

$$\mathcal{L}_{\text{OPD}} = -\mathbb{E}[a_t^{\text{detach}} \rho_t] + \tau \mathbb{E}[(\log p_{\theta}(y_t) - \log p_{\text{roll}}(y_t))^2]. \quad (7)$$

Reading it The advantage a_t asks “did the teacher like this emitted token more than the rollout policy did”; the surrogate $-a_t \rho_t$ raises the current probability of tokens with positive advantage and lowers it for negative advantage, exactly REINFORCE with the teacher as judge [Williams, 1992]. The quadratic τ term is a trust region: it penalizes the current policy for moving away from the rollout policy, keeping each update conservative (a decoupled-PPO clip additionally drops tokens whose probability has already moved past a threshold). Crucially, there is *no full-vocabulary gold KL here*—the only signal is per-token preference plus the trust region.

Why it half-works and half-breaks Targeting student-visited states is the right idea, and the sampled-token form does lift schema completion: it learns to emit the defaults the teacher would include (params: { }, unit, timeout), worth +12 at $k=160k$. But two failure modes are visible in the data and both follow from the missing distributional anchor. First, the *prefix* variant with no anchor collapses outright (36/1007 from a student rollout, 0/1007 from a base rollout): early masked rollouts are degenerate, and “match the teacher on these prefixes” simply reinforces being on degenerate prefixes; adding back an offline gold-prefix KL stops the collapse but lands below pure offline (445/487 vs 568), the generated branch dragging it down. Second, the sampled-token variant introduces *value corruption* (9.8→99.0, AAPL→Apple Inc.): on argument-value tokens the teacher–student gap is noisy, and the negative-advantage branch removes mass from the student’s emitted token without the capacity to place it on the correct value, so the rank-limited adapter resolves the deficit onto a wrong number or string. The overlap analysis confirms the trade—31 schema/default fixes bought at the price of 24 value-grounding and 12 syntax regressions. The fix is therefore not “more OPD” but “OPD with the value-preserving anchor and without the destructive pushdown.”

B.5 Post-attribution: GKD+AOPD hybrid

Algorithm From a supervised-adapter initialization, optimize the sum of the §B.3 anchor and a constrained §B.4 online term, under a zero-isolation train mask:

$$\mathcal{L} = \beta_{\text{on}} \mathcal{L}_{\text{OPD}}^+ + \beta_{\text{off}} \text{KL}_{\text{gold}} + \beta_{\text{ce}} \text{CE}, \quad (\beta_{\text{on}}, \beta_{\text{off}}, \beta_{\text{ce}}) = (0.2, 0.8, 0.05), \quad (8)$$

where $\mathcal{L}_{\text{OPD}}^+$ is the sampled-token surrogate restricted to $a_t > 0$.

Why each design choice, against the §B.4 diagnosis (i) **Positive-only online** directly removes the corruption source: the negative-advantage pushdown was what relocated value mass onto wrong tokens, so we keep only the additive half—reinforce where the teacher agrees, never subtract. (ii) **A dominant offline anchor** ($\beta_{\text{off}}=0.8$ vs $\beta_{\text{on}}=0.2$) reinstates the full-vocabulary, value-preserving gold KL that §B.4 deleted; it is the floor that holds argument values in place while the small online term adds the schema completions on the student’s own states. (iii) **Matched masking** (train under zero-isolation, the eval operator) removes a calibration error: under mean-ablation the deactivated channels inject a *constant nonzero* mean offset into the residual stream, and the adapter learns to cancel that specific offset; at evaluation the offset is zero, so a mean-ablation-trained compensation is simply mis-tuned for the lesion actually tested.

What the jump decomposes into Relative to the supervised init (422/1007), the hybrid reaches 553 and, with a heavier online weight, 562 (+131/ +140). The ablations let us attribute the gain rather than assert it. An *offline-only* hybrid—matched masking plus the gold anchor, no online term—already reaches 543/1007, so **most of the jump** ($\approx +121$) **is the matched-mask offline-policy anchor**, i.e. fixing the §B.4 anchor deletion and the §B.5(iii) mask mismatch together; the positive-only online term then adds the final $\sim +10$ –19 (543 → 553 → 562). Sweeps confirm the weights are near-optimal: raising CE to 0.10, halving the online weight, or removing it entirely all stay at or below 553. Finally, the **supervised initialization is load-bearing for the online term specifically**—OPD is only informative when the rollout policy is already competent, and a fresh-LoRA cold start scales monotonically with rank yet even fresh $r=64$ (495) sits 58 correct below the supervised-init $r=32$ hybrid (§A.5). All of §B.2–§B.5 share the post-attribution property: none can move the mask.

C Collimation techniques at a glance

§	Regime	Technique	Teacher	Student	Objective
B.1	pre-attrib.	KL-LoRA collimation	repr. base distribution	rank-32 (merged, mask)	LoRA no task loss + β KL on full model, then re-attribute $\rightarrow$ smaller mask
B.2	post-attrib.	Masked-KL res-cue	unmasked teacher outputs	masked LoRA	masked KL + CE + unmasked-KL guardrail
B.3	post-attrib.	Offline policy distillation	unmasked base policy	masked LoRA	KL on gold prefixes, no CE
B.4	post-attrib.	On-policy sampled-token	/ unmasked base	masked (rolls out)	LoRA teacher scores student-visited states; sampled-token advantage
B.5	post-attrib.	GKD+AOPD hybrid	unmasked base	masked (sup. init)	LoRA pos-only online (0.2) + offline gold KL (0.8) + CE (0.05), matched masking

The pre-attribution row trains with the mask *absent* and recomputes the mask afterward (so it can shrink the substrate); every post-attribution row trains with the mask *present* and inherits it fixed (so it cannot).

D Method Details

ReLP implementation detail The ReLP backward pass is made locally linear by freezing nonlinear terms rather than differentiating through them: the RMSNorm denominator is held constant, the SiLU gate’s sigmoid is detached, the attention softmax weights are frozen, and the gated gate-up product is split by a half-rule. These choices matter for reproducing the attribution, but the conceptual role is just to assign one relevance score to each MLP channel.

Destructive attribution ReLP scores a channel by its *gradient-side* relevance; it does not directly measure what breaks when the channel is removed. To rank necessity *causally* within an already-selected substrate, we use a second, ablation-based procedure. Order the k kept channels by ReLP rank and partition them into contiguous chunks (in practice $k = 240,000$ split into 24 chunks of 10,000). For each chunk j , remove it from the kept set—leaving k minus one chunk active under the same deactivation operator—and re-evaluate. The chunk’s **damage** is

$$D_j = \text{score}(m_k) - \text{score}(m_k \setminus \text{chunk}_j), \quad (9)$$

the drop in masked task score caused by deleting it. Positive D_j marks a chunk that the rest of the circuit *depends on*; negative D_j marks one that is redundant or actively harmful to keep. Ranking chunks by D_j yields a destruction-ranked attribution that is complementary to ReLP: ReLP says where relevance *flows*, destruction says what the behavior cannot *survive without*. We run it on collimated models to ask whether a smaller sufficient core hides inside the substrate (§A.4); a key property we test is whether per-chunk damages *compose*—whether dropping several individually-harmful chunks together helps, as additivity would predict, or not.

Collimation data Collimation is a training procedure, so the data it runs on is part of the method. If the data distribution is wrong, the adapter can shift the model instead of reshaping the target capability. Two principles emerged. First, the collimator must train on the *capability’s own distribution*, not a convenient proxy. For arithmetic we collimate on exhaustive 2-digit sums; for function calling we mine a strict, schema-validated single-call corpus. The v2 *strict mix* has 10k rows: 6,821 from ToolMind [Yang et al., 2025] and 3,179 from Argilla/APIGen [Liu et al., 2024]. Each row has one user prefix, exactly one tool, and one target call whose function and arguments validate against the schema; BFCL is held out entirely for evaluation. A rougher mix is actively harmful. *Repair-v1*, a generic ToolMind-plus-synthetic-hotfix blend, raised the masked circuit by +3.5 points while degrading the *unmasked* model by −4.6, so it moved global behavior rather than rerouting the target capability. Second, synthetic teacher data carries a measurement hazard we document but do not fully solve. When references are themselves LLM-generated, as in our DeepSeek-built EN→PT corpus, both the teacher and the reference gravitate to the same fluent register. Soft-judgement failures are

then *under-counted*: terminology gaps surface at 47.8%, while register/idiom/factual buckets pass at 74–79%, materially easier than the real FLORES baseline [NLLB Team et al., 2022]. Collimation data must therefore be capability-matched *and* validated against a non-synthetic signal where one exists.

E Artifacts and Reproducibility

Claim	Result	Contract	Receipt
Arithmetic sparse substrate	12,661 MLPs, 91.33% autoregressive	CF-patch MLP mask, fixed trans-former scaffold	issue #8; results/qwen25_math_1p5b_2digit_*/summary.md
Arithmetic high recovery	58,619 MLPs, 99.53% autoregressive	CF-patch MLP mask, fixed trans-former scaffold	issue #8; rank-32 validation artifacts
Function-calling raw mask	k160: 123/1007; k240: 396/1007	zero-isolated MLP mask; strict single-call BFCL slice	issue #49; runs/issue49_bfcl_single_call/relp_full.npz
Function-calling collimated	k160: 562/1007; k240: 568/1007	normalized exact; same MLP mask, fixed scaffold	issues #49–#51; policy_distill_*, prime_opd_k160_*
Function-calling packing	negative chunks do not compose; k240 repair does not fold into k160	post-attribution collimator under frozen mask	issue #49; k240_r16_v2_normalized_destruction_attr.npz
Translation broad substrate	ReLP k160 is the honest EN→PT boundary	mean-ablation MLP mask; broad substrate, not tiny MVC	issues #34, #37, #48
Translation rescue	rank-32 conservative floor at k120	fixed-lesion rescue, not replacement	issue #24; results/issue24_enpt_rescue/summary.md

Table 4: Claim-to-receipt map. Each headline result is tied to its denominator, intervention contract, and artifact trail.

Known reproducibility gaps The BFCL result uses a stricter normalized exact scorer than the official BFCL harness; absolute scores should be read under this stricter contract. Issue #49 remains open in the experiment ledger because the official scorer, stable dataset package, and artifact index are not yet normalized. Translation reporting keeps necessity, rescue, and sufficiency separate because later chrF++ audits [Popović, 2017] corrected earlier XCOMET-only destructive interpretations. The machine-readable experiment ledger is docs/experiment_issue_ledger.json.

F Extended Related Work

Sparsity, compression, and adaptation Pruning and sparse-training methods reduce capacity by removing or gating units while preserving model-level behavior [Louizos et al., 2018]. Knowledge distillation trains a smaller or constrained student against a larger teacher’s distribution [Hinton et al., 2015, Kim and Rush, 2016], and LoRA supplies a low-rank parameter-efficient way to adapt a model [Hu et al., 2022]. Prism uses these ingredients for a different object. The goal is not merely a cheaper model or a better adapter; it is a capability that remains recoverable through a specified sparse MLP-channel substrate after the complement is deactivated.

Attribution and native-basis sparsity Causal mediation, activation patching, path patching, and automated circuit discovery identify components whose ablation changes a behavior [Vig et al., 2020, Wang et al., 2023, Goldowsky-Dill et al., 2023, Conmy et al., 2023]. These methods supply ways to score and test internal components. Our attribution is ReLP, the relevance-propagation method of Arora et al. [2026]: an exact forward pass with a linearised backward pass that scores every MLP intermediate channel by its contribution to the teacher-forced log-probability of a target continuation in a single pass (§2). We treat attribution as the *first* of two stages. The second stage is collimation, which changes whether the scored channels can form an efficient recoverable substrate.

Polysemanticity and superposition A central obstacle for native-basis attribution is that individual units are polysemantic and features are held in superposition, so a capability need not be sparse along the neuron axes [Elhage et al., 2022, Bricken et al., 2023]. Sparse autoencoders attack this by learning a sparser *feature* basis post hoc [Cunningham et al., 2023, Bricken et al., 2023]; we instead change the model’s own basis by collimation, so the capability becomes extractable along channels the standard intervention can keep.

On-policy distillation Generalized knowledge distillation (GKD; Agarwal et al., 2024) uses student-generated, on-policy sequences to address train/inference mismatch. We adapt this idea as an *intra-model* objective: the teacher is the unmasked base model and the student is the same model under a sparse mask, distilling the model’s own policy into a bounded substrate of itself.

Evaluation Translation is scored with learned metrics XCOMET-XXL and COMET [Guerreiro et al., 2024, Rei et al., 2020]. Function calling is scored on the Berkeley Function Calling Leaderboard (BFCL v3; Patil et al., 2025), used as a held-out data source under a stricter exact-match than the official harness.

G Extended Discussion

The two regimes are two faces of one efficiency claim, but they are not interchangeable, and the difference is the *order of collimation and attribution*, not a difference between the tasks. Arithmetic collimation is **pre-attribution and global**: it reshapes the full model and we re-attribute, so the sufficient mask shrinks an order of magnitude while behavior is held fixed. Function-calling collimation is **post-attribution and masked**: the channel set is frozen before the adapter trains, so recovery is governed entirely by how the rerouting adapter is trained and the mask will not move. We emphasize that this is a property of the two *protocols we ran*, not a property of arithmetic versus function calling. We have not run pre-attribution collimation on function calling, and nothing here implies its frontier is immovable—only that no post-attribution adapter can move it. In both protocols, the controlled variable that determines extractability is the collimation technique, not the attribution score or the channel budget.

This has a methodological consequence for efficiency claims. Reports of “we found / did not find sparsity” are under-specified unless they fix (a) the collimation objective and (b) the intervention contract. The $4\times$ rescue/replacement gap at an identical, intact region (§5.3) shows two defensible readings of “using the same sparse region” can differ by a factor of four on the same channels. Pinning the contract before the substrate resolves much apparent disagreement.

Practically, the policy-distillation result suggests a recipe for *capability-level self-compression*: a model’s own unmasked policy is a stronger teacher for a masked substrate than external labels, and a constrained online/offline hybrid is the stable way to exploit student-visited states without collapse.

H Extended Limitations and Future Work

A *post-attribution* collimator cannot shrink a fixed substrate (§A.4); the highest-value untested experiment is therefore to port the *pre-attribution* arithmetic protocol to function calling—KL-collimate the full unmasked Qwen3-8B on the tool-calling policy, merge, and re-attribute—to see whether its frontier moves as arithmetic’s did; our results neither demonstrate nor preclude it. Failing that, genuine size reduction would require reshaping the budget *during* attribution/training (e.g. an L0 or gating objective [Louizos et al., 2018]) rather than a better post-hoc adapter. Function-calling recovery saturates near 85% because every objective past the structural fix trades value grounding against schema/default completion; closing this needs schema-completion-specific repair signal or an objective that holds both buckets.

Arithmetic recovery is reported under a causal mask with intervention scaffolding, not as a standalone model, and the strongest claims are on a math-tuned 1.5B model where pretraining already favors sparsity. The model-dependence account is a hypothesis, not yet a causal intervention: HY-MT behaves as if heavy distillation has spent the slack collimation needs, but the direct test is to compare matched models across specialization strength. Function-calling exact-match is stricter than the official BFCL harness, so absolute scores understate official accuracy; relative comparisons across collimators (the paper’s claims) are unaffected. Whether the collimated arithmetic mask exposes a pre-existing circuit, reweights one, or learns a new sparse shortcut remains open and is the subject of transfer and causal-projection controls.

Conference For AI Scientists 2026 - AI Involvement Checklist

This checklist documents the role of AI systems in the research process. The checklist has two parts: Research Stage Assessment and AI System Documentation.

Research Stage Assessment

For items 1–4, the involvement scale is:

- **[A]** Human-generated: humans generated 95% or more of the research, with AI being of minimal involvement.
- **[B]** Mostly human, assisted by AI: the research was a collaboration between humans and AI models, but humans produced the majority.
- **[C]** Mostly AI, assisted by human: the research task was a collaboration between humans and AI models, but AI produced the majority.
- **[D]** AI-generated: AI performed over 95% of the research.

For stages where AI was involved, the iteration effort scale is:

- **[Low]** approximately 1–10 substantive attempts.
- **[Medium]** approximately 10–100 substantive attempts, with some failed attempts or course corrections.
- **[High]** approximately 100+ substantive attempts, with substantial exploration, failures, or trial-and-error.
- **[NA]** not applicable because AI involvement was **[A]**.
- **[Unclear]** the authors cannot reasonably estimate.

1. **Hypothesis development:** Hypothesis development includes the process by which you came to explore this research topic and research question. This can involve the background research performed by either researchers or by AI. This can also involve whether the idea was proposed by researchers or by AI.

Answer: **[B]**

Iteration Effort: **[Medium]**

Explanation: Human researchers set the high-level direction around capability extraction, sparsity, and conditioning. AI systems assisted with literature framing, adversarial review of claims, terminology, and conversion of informal hypotheses into testable experimental contracts.

2. **Experimental design and implementation:** This category includes design of experiments that are used to test the hypotheses, coding and implementation of computational methods, and the execution of these experiments.

Answer: **[C]**

Iteration Effort: **[High]**

Explanation: AI coding and research agents assisted substantially in implementing attribution, masking, conditioning, scorer normalization, artifact checks, and experiment-ledger workflows. Human researchers selected the research direction, inspected failures, chose follow-up experiments, and decided which claims survived.

3. **Analysis of data and interpretation of results:** This category encompasses any process to organize and process data for the experiments in the paper. It also includes interpretations of the results of the study.

Answer: **[C]**

Iteration Effort: **[High]**

Explanation: AI systems assisted with aggregating experiment results, comparing recovery regimes, identifying failure buckets, and stress-testing alternative interpretations. Human researchers made the final judgment about claim boundaries, terminology, and which objections were contract failures versus genuine limitations.

4. **Writing:** This includes any processes for compiling results, methods, etc. into the final paper form. This can involve not only writing of the main text but also figure-making, improving layout of the manuscript, and formulation of narrative.

Answer: [C]

Iteration Effort: [High]

Explanation: AI writing and coding agents assisted heavily with drafting, restructuring, bibliography checks, figure integration, LaTeX compilation, and reviewer-style critique. Human researchers controlled the paper's positioning, claim appetite, final framing, and acceptance or rejection of proposed edits.

AI System Documentation

5. **AI systems used:** Describe all AI systems used in this research without naming specific products or models. Use system-level descriptors only. Include relevant details such as system type, scale, and capabilities. Specific model and product names should only be included in the camera-ready version.

Description: The work used frontier large-language-model research assistants, terminal-based coding agents, and document-editing agents capable of reading code, editing LaTeX, running experiments, inspecting logs, and synthesizing paper prose. The experimental subject models and benchmark artifacts are named in the main text because they are part of the scientific evaluation rather than the authoring system.

6. **Observed AI Limitations:** What specific limitations and failure modes have you observed when using AI as a partner or lead author?

Description: AI systems were useful for implementation, summarization, and adversarial review, but they required human correction for overconfident framing, stale or incomplete citation assumptions, overly timid scientific language, and occasional mismatch between a formal claim and the intended operational contract. Long experiment histories also required explicit artifact-level verification rather than relying on agent memory.

Conference For AI Scientists 2026 - Reproducibility and Responsibility Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction state the extraction contract, the two conditioning regimes, and the measured recovery results. The limitations section states that arithmetic recovery is not a standalone compressed model and that pre-attribution conditioning for function calling remains untested.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Section H discusses untested pre-attribution function-calling conditioning, the causal-mask rather than standalone-compression boundary, model dependence, scorer strictness, and unresolved shortcut versus pre-existing-circuit interpretations.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate limitations section.
- The paper should point out strong assumptions, robustness limits, scope of claims, computational efficiency, and relevant privacy or fairness limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[NA\]](#)

Justification: The paper is empirical and methodological. It does not present formal theorems or proofs.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All theorems, formulas, and proofs should be numbered and cross-referenced when applicable.
- All assumptions should be clearly stated or referenced in the statement of any theorem.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper, regardless of whether the code and data are provided or not?

Answer: [\[Yes\]](#)

Justification: The paper specifies models, datasets, intervention operators, metrics, conditioning objectives, denominators, and claim-to-artifact receipts in the appendix. Known gaps around the BFCL official scorer and artifact normalization are explicitly disclosed.

Guidelines:

- The answer NA means that the paper does not include experiments.
- Making the paper reproducible is important, and a No answer for experimental papers will be viewed negatively.
- Closed or difficult-to-share artifacts should still leave a path for reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [\[Yes\]](#)

Justification: An anonymized code-and-data package is provided as supplementary material. It contains the attribution, masking, and conditioning code for all three domains; the arithmetic and translation result receipts; partial function-calling results; documented public data sources (BFCL, NTREX-128, ToolMind, APIGen) with regeneration scripts for the generated arithmetic and strict tool-call mixes; and an anonymization and leak-scan report. A subset of function-calling run artifacts—the raw attribution file, the exact 1007-row evaluation slice, and the strict-mix training data behind the headline BFCL numbers—is not yet included and is listed for later release, consistent with the BFCL artifact-normalization gap disclosed above.

Guidelines:

- The answer NA means that the paper does not include experiments requiring code.
- Papers cannot be rejected simply for not including code, unless code release is central to the contribution.
- At submission time, authors should release anonymized code and data when applicable.

6. Experimental setting/details

Question: Does the paper specify all the training and test details necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: Sections 4, B, and the appendix specify models, datasets, scorer variants, mask sizes, intervention operators, conditioning losses, rank choices, and held-out evaluation settings.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented to the level necessary to understand the results.
- Full details may be provided in the main paper, appendix, code, or supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[Yes\]](#)

Justification: Function-calling exact match is a binomial outcome over the fixed 1007-row BFCL slice; Appendix A.7 (Figure 4) reports 95% Wilson score intervals on every masked exact-match proportion, showing that the conditioning gains exceed evaluation-set sampling uncertainty while some adjacent conditioners are statistically indistinguishable. These intervals quantify sampling uncertainty on the fixed evaluation slice. Arithmetic and translation are reported as point estimates on exhaustive or held-out sets, and training-seed variance is not estimated because the reported conditioner comparisons are single-seed runs.

Guidelines:

- The answer NA means that the paper does not include experiments.

- Error bars, confidence intervals, or statistical significance tests should accompany experiments where variability affects the main claims.
- The factors of variability captured by error bars should be clearly stated.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources needed to reproduce the experiments?

Answer: [\[Yes\]](#)

Justification: All experiments adapt 1.5B–8B-parameter models with low-rank adapters (LoRA rank 4–64, batch size 16, 3 epochs, ≤ 300 steps) on a single multi-GPU node (typically four GPUs); attribution and masked evaluation are single- or few-pass and run in seconds to minutes per point. We estimate total project compute at the order of 10^3 H100-equivalent GPU-hours—roughly US\$3,000 at current on-demand cloud rates—dominated by the 8B function-calling conditioning runs. Exact per-run wall-clock was not logged uniformly across runs, so this is a cost-normalized estimate rather than a measured per-experiment bill.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate CPU/GPU type, memory, storage, and execution time where relevant.
- The paper should provide per-run and total compute estimates when possible.

9. Research Ethics

Question: Does the research conducted in the paper conform with the NeurIPS Code of Ethics, except where CAISc’s AI authorship policies apply?

Answer: [\[Yes\]](#)

Justification: The work studies model-internal sparsity and capability extraction on standard model and benchmark artifacts. The submission follows CAISc’s AI-involvement disclosure requirement and does not involve human-subject data collection.

Guidelines:

- All submissions should adhere to the NeurIPS Code of Ethics, except where CAISc’s AI authorship policies apply.
- The answer NA means that the authors have not reviewed the code of ethics.
- If the authors answer No, they should explain the circumstances requiring a deviation.

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [\[Yes\]](#)

Justification: The main paper discusses the positive use case of capability-level efficiency and precise intervention tests, and the negative possibility that successful extraction could make narrow behaviors easier to isolate and reuse.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why.
- Potential negative impacts include malicious or unintended uses, fairness, privacy, and security considerations.